

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЕТ
по дисциплине
«Основы систем мобильной связи»

по теме:
РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА. РЕАЛИЗАЦИЯ ПРИЕМА И
ПЕРЕДАЧИ БИТОВОЙ ПОСЛЕДОВАТЕЛЬНОСТИ В УСЛОВИЯХ
ПОМЕХ.

Студент:
Группа № ИА331

В.А. Павлова

Предподаватель:
В.Г. Дроздова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА.....	4
1.1	Цель работы	4
1.2	Задачи лабораторной работы	4
2	КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	6
2.1	Цифровая модуляция и передача данных	6
2.1.1	Принцип амплитудной манипуляции (ASK).....	6
2.1.2	Аддитивный белый гауссовский шум (AWGN)	6
2.2	Методы обеспечения надёжности передачи	7
2.2.1	Циклический избыточный код (CRC)	7
2.2.2	Синхронизация и последовательность Голда.....	7
2.3	Детектирование сигнала и восстановление данных	8
2.3.1	Пороговое детектирование	8
2.3.2	Спектральный анализ цифровых сигналов	9
3	ХОД РАБОТЫ	10
3.1	Введем с клавиатуры имя и фамилию латиницей.	10
3.2	Сформируем битовую последовательность.....	10
3.3	Вычислим CRC	12
3.4	Последовательность Голда	13
3.5	Преобразуем биты с данными во временные отсчеты сигналов	14
3.6	Формирование второго массива.....	15
3.7	Добавление шумов	16
3.8	Реализация функции корреляционного приема	17
3.9	Разбор зашумленного сигнала на биты	18
3.10	Удаление синхропоследовательности	18
3.11	Проверка CRC.....	19
3.12	Декодирование	19
3.13	Визуализация переданного и полученного сигнала	20
4	ВЫВОДЫ	21
4.1	РЕЗУЛЬТАТЫ	21
4.2	ГИТХАБ	22

5	СКРИПТЫ	23
5.1	Скрипт RGR	23

1 РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА

1.1 Цель работы

Закрепить и структурировать знания, полученные в рамках изучения дисциплины «Основы систем мобильной связи».

1.2 Задачи лабораторной работы

1. Ввести с клавиатуры имя и фамилию латиницей.
2. Сформировать битовую последовательность, кодирующую введенный текст в ASCII. Разработать ASCII-кодер, вывести массив битов и визуализировать его на графике.
3. Вычислить CRC заданной длины для битовой последовательности, используя параметры из варианта работы №5. Разработать CRC-генератор, вывести контрольную сумму в терминал и добавить её к данным.
4. Добавить к полученной последовательности (данные + CRC) преамбулу в виде последовательности Голда для синхронизации. Реализовать функцию генерации последовательности Голда, сформировать итоговый массив и визуализировать его.
5. Преобразовать результирующую битовую последовательность во временные отсчеты сигнала методом амплитудной манипуляции (Amplitude-Shift Keying, ASK), отводя на каждый бит по N отсчетов. Визуализировать полученный массив временных отсчетов.
6. Сформировать «кадр» передачи, поместив массив отсчетов из п.5 в нулевой массив удвоенной длины, начиная с позиции, заданной пользователем. Визуализировать итоговый массив Signal.
7. Имитировать прохождение сигнала через канал с аддитивным белым гауссовским шумом (AWGN). Сгенерировать массив шума по нормальному распределению с заданной дисперсией, сложить с сигналом и визуализировать зашумленный принятый сигнал.
8. Реализовать корреляционный приёмник для обнаружения момента начала синхросигнала (последовательности Голда) в зашумленном

сигнале. Определить и вывести стартовый отсчёт, обрезать лишние биты до него.

9. Реализовать функцию порогового детектирования. На основе накопления энергии по N отсчётам и сравнения с порогом P восстановить битовую последовательность $(L+M+G)$ бит из оцифрованного сигнала.
10. Удалить из восстановленной битовой последовательности G бит синхронизации (преамбулу Голда).
11. Проверить корректность приёма оставшихся данных $(L+M)$ бит путём пересчёта CRC. Вывести в терминал результат проверки (наличие или отсутствие ошибок).
12. В случае отсутствия ошибок по CRC, удалить биты CRC и декодировать оставшиеся L бит с помощью ASCII-декодера. Вывести восстановленный текст на экран.
13. Провести спектральный анализ:
 - Визуализировать спектры передаваемого и зашумлённого принятого сигналов.
 - Исследовать влияние длительности символа на спектр: получить и отобразить на одном графике спектры сигналов с исходной длительностью символа, уменьшенной в два раза и увеличенной вдвое.
14. Сформулировать промежуточные выводы по каждому этапу обработки сигнала.
15. Оформить отчёт.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Цифровая модуляция и передача данных

2.1.1 Принцип амплитудной манипуляции (ASK)

Амплитудная манипуляция (Amplitude-Shift Keying, ASK) — один из простейших методов цифровой модуляции, при котором информация кодируется изменением амплитуды несущего колебания. В двоичном случае (2-ASK или OOK — On-Off Keying) логической «1» соответствует наличие сигнала определённой амплитуды, а логическому «0» — его отсутствие или значительно меньшая амплитуда.

При переходе к дискретному представлению (цифровые отсчёты), каждый бит продолжительностью T_b представляется последовательностью из N отсчётов с постоянным уровнем:

$$s[n] = \begin{cases} A, & \text{если бит} = 1 \\ 0, & \text{если бит} = 0 \end{cases}$$

для $n = 0, 1, \dots, N - 1$, где A — амплитуда сигнала, а $N = T_b \cdot f_s$, f_s — частота дискретизации.

2.1.2 Аддитивный белый гауссовский шум (AWGN)

Модель аддитивного белого гауссовского шума (Additive White Gaussian Noise, AWGN) является фундаментальной для анализа цифровых систем связи. Шум считается:

- **Аддитивным** — складывается с полезным сигналом: $r(t) = s(t) + n(t)$.
- **Белым** — его спектральная плотность мощности постоянна в бесконечной полосе частот.
- **Гауссовским** — мгновенные значения шума подчиняются нормальному (гауссовскому) распределению с нулевым средним ($\mu = 0$) и дисперсией σ^2 .

В дискретной форме принятый зашумлённый сигнал:

$$r[k] = s[k] + n[k], \quad n[k] \sim \mathcal{N}(0, \sigma^2)$$

Отношение сигнал/шум (SNR) является ключевым параметром, определяющим вероятность ошибки при приёме.

2.2 Методы обеспечения надёжности передачи

2.2.1 Циклический избыточный код (CRC)

Циклический избыточный код (Cyclic Redundancy Check, CRC) — техника обнаружения ошибок, основанная на циклических кодах. Передаваемая битовая последовательность (данные) трактуется как коэффициенты полинома $D(x)$. Генераторный полином $G(x)$ степени M известен и передатчику, и приёмнику.

1. К полиному данных $D(x)$ (степени $L - 1$) добавляется M нулевых бит, что соответствует умножению на x^M : $x^M \cdot D(x)$.
2. Полученный полином делится по модулю 2 на генераторный полином $G(x)$: $x^M \cdot D(x) = Q(x) \cdot G(x) + R(x)$, где $R(x)$ — остаток (степени $M - 1$).
3. Остаток $R(x)$, представленный в виде M -битной последовательности, является контрольной суммой CRC, которая передаётся вслед за данными.

На приёмной стороне процедура повторяется для принятых данных вместе с CRC. Если остаток от деления не равен нулю, при передаче произошла ошибка.

2.2.2 Синхронизация и последовательность Голда

Корректный приём цифрового потока требует точного знания момента начала каждого символа (битовой синхронизации) и момента начала кадра данных (кадровой синхронизации).

Последовательность Голда — это псевдослучайная бинарная последовательность (ПСП), обладающая хорошими автокорреляционными свойствами: её автокорреляционная функция имеет выраженный острый пик при нулевом сдвиге и низкий уровень боковых лепестков. Это делает её идеальным кандидатом на роль синхропоследовательности (преамбулы).

На приёмной стороне для поиска начала кадра используется **корреляционный приёмник**. Вычисляется взаимная корреляция между принятым зашумлённым сигналом $r[k]$ и известным шаблоном синхропоследовательности $g[k]$:

$$R_{rg}[m] = \sum_{k=0}^{G \cdot N - 1} r[k + m] \cdot g[k]$$

Индекс m_{peak} , соответствующий максимальному значению корреляции $R_{rg}[m]$, указывает на наиболее вероятное начало синхросигнала и, соответственно, начало полезных данных.

2.3 Детектирование сигнала и восстановление данных

После синхронизации и выделения участка сигнала, соответствующего одному биту (длиной N отсчётов), необходимо принять решение, был ли передан «0» или «1».

2.3.1 Пороговое детектирование

При использовании ASK модуляции детектирование можно свести к оценке энергии сигнала на интервале бита:

$$E = \sum_{i=0}^{N-1} |s_i|^2$$

где s_i — отсчёты сигнала на интервале детектирования.

Решение принимается путём сравнения E с порогом P :

$$\hat{bit} = \begin{cases} 1, & \text{если } E \geq P \\ 0, & \text{если } E < P \end{cases}$$

Оптимальный порог P_{opt} в условиях AWGN и равновероятных битов находится как среднее между энергиями, соответствующими «0» и «1». Если «0» передаётся как отсутствие сигнала, то $P_{opt} \approx \frac{A^2 \cdot N}{2}$.

2.3.2 Спектральный анализ цифровых сигналов

Спектр дискретного сигнала $s[n]$ длиной L_{total} отсчётов можно оценить с помощью Быстрого Преобразования Фурье (БПФ):

$$S[f] = \sum_{n=0}^{L_{total}-1} s[n] \cdot e^{-j2\pi fn/L_{total}}$$

Анализ спектра позволяет:

- Оценить занимаемую сигналом полосу частот.
- Наблюдать влияние шума (спектр зашумлённого сигнала $R[f]$ будет «приподнят» по всему частотному диапазону на величину, связанную с σ^2).
- Исследовать влияние длительности символа T_b на ширину спектра. Чем короче символ (выше скорость передачи), тем шире спектр. Увеличение N при постоянной f_s эквивалентно увеличению T_b и приводит к сужению основного лепестка спектра.

3 ХОД РАБОТЫ

Исходные данные

- **Номер группы:** нечетный (используется схема с рисунка 4.5)
- **Номер в журнале:** 12
- Порождающий полином: 11011110
- Длина CRC-кода: 7 бит
- Входные данные (имя, Фамилия): Viktoria Pavlova

3.1 Введем с клавиатуры имя и фамилию латиницей.

Реализуем считывание данных с клавиатуры в матлаб:

```
1 name = input('Имя латиницей: ', 's');  
2 surname = input('Фамилия латиницей: ', 's');  
3  
4 fprintf('\Авторn работы: %s %s\n\n', name, surname);
```

```
>> PGP  
Имя латиницей: Vika  
фамилия латиницей: Pavlova  
  
Автор работы: Vika Pavlova
```

Рисунок 1 — Ввод ФИ

3.2 Сформируем битовую последовательность

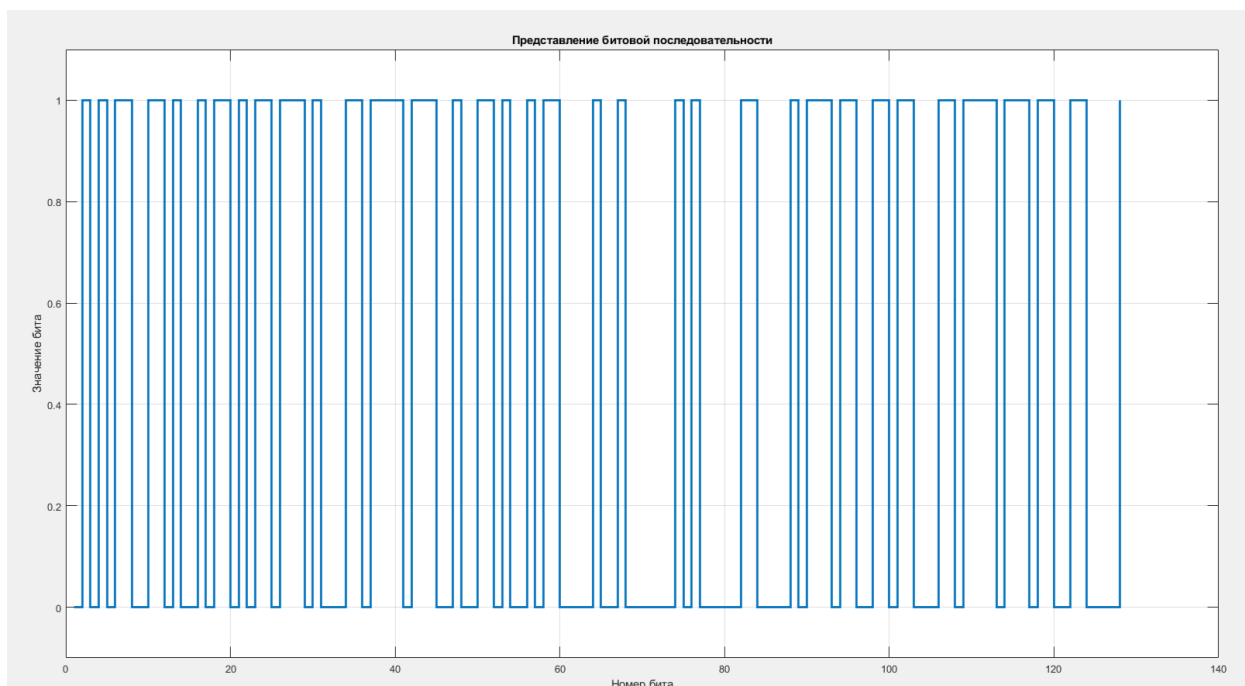
Для того чтобы сформировать битовую последовательность, реализуем ASCII-кодер. Функция `double()` преобразует строку в массив числовых значений, где каждый элемент соответствует ASCII-коду соответствующего символа. Для латинских символов используются коды в диапазоне от 0 до 255. Функция `dec2bin()` преобразует массив ASCII-кодов в двоичные строки фиксированной длины 8 бит, что соответствует стандартному размеру байта

в ASCII-кодировке. Используется математическая операция вычитания символа '0', которая преобразует символы '0' и '1' в числа 0 и 1 соответственно. Функция `reshape()` преобразует полученную матрицу битов в одномерный массив, обеспечивая последовательное расположение битов всех символов. Для проверки декодируем полученную битовую последовательность обратно в символы:

```
1 text = [name ' ' surname];
2 fprintf('Исходный текст: "%s"\n', text);
3
4 % ASCII-кодер-
5 bits = dec2bin(double(text), 8) - '0';
6
7 % Преобразование в одномерный массив
8 res = reshape(bits', 1, []);
9 L = length(res);
10 % Декодирование для проверки
11 fprintf('\Проверка декодированием: "%s"\n', char(bin2dec(reshape(char(res' +
    ↪ '0'), 8, []))));
```

```
>> PGP
Имя латиницей: Viktoria
Фамилия латиницей: Pavlova
Исходный текст: "Viktoria Pavlova"
Длина последовательности L = 128 бит
Битовая последовательность:
010101100110100101101011011101000110111101110010011010010110000100100000010100000110000
Проверка декодированием: "Viktoria Pavlova"
```

Рисунок 2 — ASCII-кодер



3.3 Вычислим CRC

CRC — это метод обнаружения ошибок, основанный на вычислении контрольной суммы для проверки целостности данных. В основе алгоритма лежит операция деления полиномов в двоичном формате. Используется порождающий полином 11011110, соответствующий выражению $G(x) = x^7 + x^6 + x^4 + x^3 + x^2 + x$. Длина контрольной суммы составляет 7 бит.

Алгоритм работает следующим образом:

К исходной битовой последовательности дописывается 7 нулей

Выполняется деление полученной последовательности на порождающий полином с помощью операций XOR

Остаток от деления (7 бит) добавляется к исходным данным

На приёмной стороне выполняется аналогичное деление полной последовательности (данные + CRC). Нулевой остаток подтверждает корректность передачи. Данный метод обеспечивает обнаружение одиночных, двойных и любого нечётного количества ошибок в пакете.

```

>> FGP
Имя латиницей: viktoriia
фамилия латиницей: pavlova
Исходный текст: "viktoriia pavlova"
Длина последовательности L = 136 бит
Битовая последовательность:
011101100110100101101011011010001101111011100100110100101101001011000010010000001110000011000010111011001101100011011110111011001100001

CRC-код (7 бит): 1010100

Данные с CRC (143 бит):|
0111011001101001011010110110100011011110111001001101001011010010110000100100000011100000110000101110110011011000110111101110110011000011010100

Проверка CRC: OK (без ошибок)

Проверка декодированием: "viktoriia pavlova"

```

Рисунок 4 — crc

3.4 Последовательность Голда

Последовательность Голда используется для синхронизации между передатчиком и приемником, то есть является ”маркером” начала полезных данных. Последовательность Голда формируется путем сложения по модулю 2 двух М-последовательностей, сгенерированных на основе заданных начальных состояний регистров ($x=12$, $y=19$).

- Регистр X: $x_{\text{new}} = x_3 \oplus x_5$
- Регистр Y: $y_{\text{new}} = y_3 \oplus y_5$
- Последовательность Голда: $X \oplus Y$

Данная последовательность длиной 31 бит обладает оптимальными автокорреляционными свойствами. Приемник осуществляет поиск этой известной последовательности в принимаемом сигнале методом скользящей корреляции, что позволяет точно определить момент начала полезных данных. После обнаружения синхропоследовательности приемник извлекает последующие биты, содержащие информационные данные и контрольную сумму CRC.

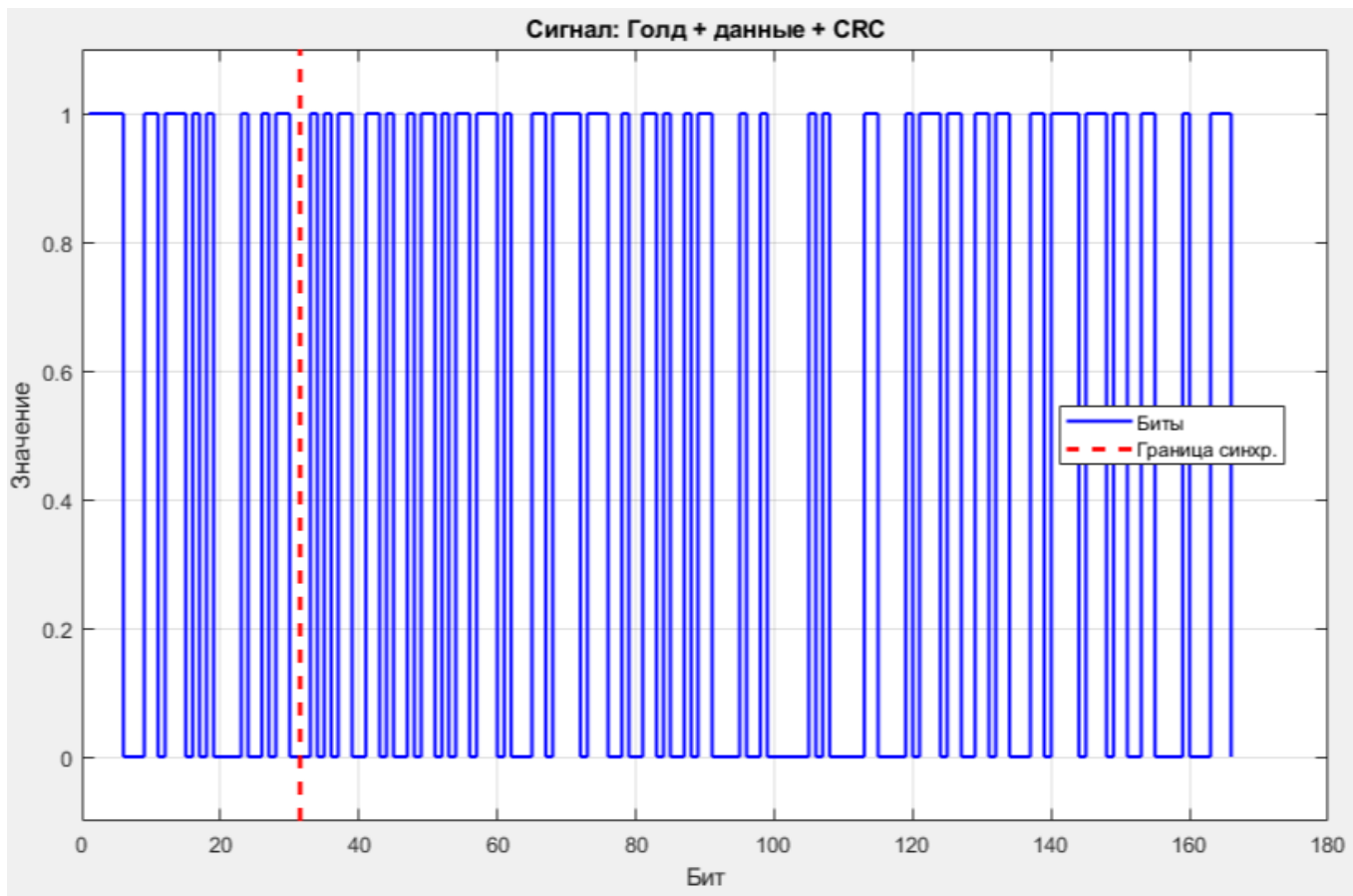


Рисунок 5 — полученная битовая последовательность с последовательностью Голда и контрольной суммой

```
Последовательность Голда (31 бит):
1111100011011101010000100101100
```

Рисунок 6 — полученная последовательность Голда

3.5 Преобразуем биты с данными во временные отсчеты сигналов

Для преобразования дискретной битовой последовательности в непрерывный временной сигнал реализуем апсемплинг. Возьмем $N = 5$. Каждый бит заменили на N одинаковых отсчетов. Бит 0 $\rightarrow$ N нулей, бит 1 $\rightarrow$ N единиц. Получили непрерывный сигнал для передачи по каналу.

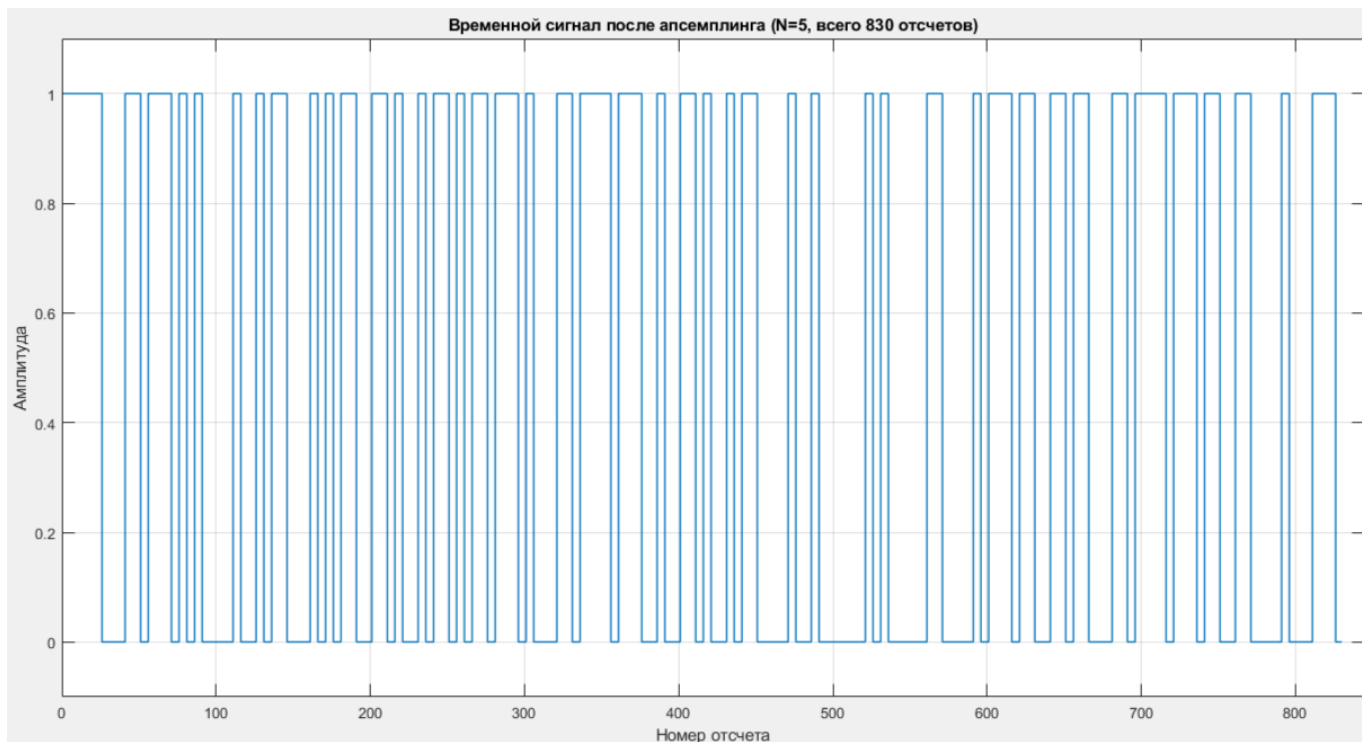


Рисунок 7 — полученная битовая последовательность после апсемплинга

```

АПСЕМПЛИНГ
Битов: 166
Отсчетов на бит: 5
Всего отсчетов: 830
Первые 50 отсчетов:
11111 11111 11111 11111 11111
00000 00000 00000 11111 11111

```

Рисунок 8 — результаты апсемплинга

3.6 Формирование второго массива

Создадим нулевой массив длиной $2 \times N \times (L+M+G)$. Напишем функции для ввода числа с клавиатуры. В соответствии с введенным числом вставим в массив значения полученный после апсемплинга. так же пропишем проверку на корректность введенного числа.

```

1 reserved = zeros(1, 2*N*(M+L+G));
2 fprintf('\Длина второго массива: %d бит', length(reserved));
3
4 position = input(sprintf('Введите позицию для вставки (0-%d): ', (N*(M+L+G))));
5 start_idx = position + 1;

```

```

6 end_idx = min(position + length(up_signal), length(reserved));
7
8 % Проверяем, чтобы не выйти за границы массива
9 if start_idx <= length(up_signal)
10     reserved(start_idx:end_idx) = up_signal(1:(end_idx-start_idx+1));
11 else
12     fprintf('Ошибка: позиция выходит за границы массива\n');
13 end

```

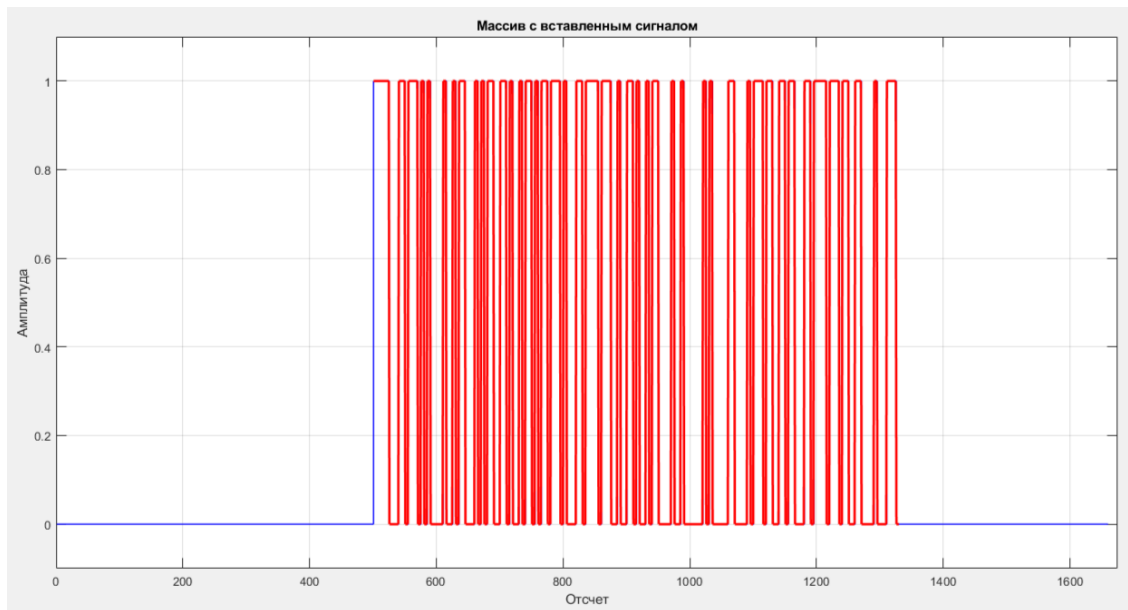


Рисунок 9 — полученная битовая последовательность после вставки

```

Длина второго массива: 1660 бит
Введите позицию для вставки (0-830): 500

```

Рисунок 10 — результаты

3.7 Добавление шумов

Для моделирования реальных условий передачи сигнала по радиоканалу сгенерируем аддитивный белый гауссовский шум (AWGN). Шум формируется по нормальному распределению с нулевым математическим ожиданием ($\mu=0$) и стандартным отклонением σ , задаваемым с клавиатуры. Массив шума той же длины, что и передаваемый сигнал, создается функцией `randn`, которая генерирует случайные числа с распределением $N(0,1)$, после чего

умножается на коэффициент σ . Сформированный шум поэлементно складывается с исходным сигналом, имитируя искажения при прохождении через канал связи. Получили зашумленный сигнал с параметрами $\sigma = 0.3$, $\mu=0$

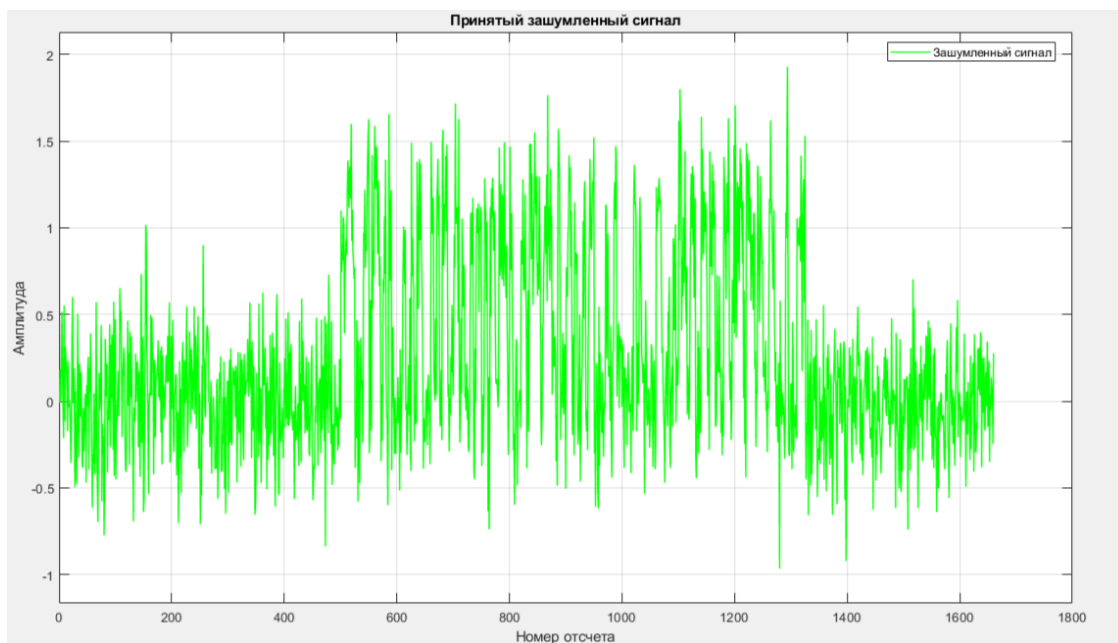


Рисунок 11 — Зашумленный принятый сигнал

3.8 Реализация функции корреляционного приема

Для реализации нам необходимо ранее посчитанную последовательность Голда отправить на апсемплинг. Далее считаем нормированную корреляцию между принятым зашумленным сигналом и последовательностью Голда. Где корреляция максимальна — там начало нашего сигнала. Отрезаем всё до этой точки, выводим номер отсчета, где начинаются полезные данные.

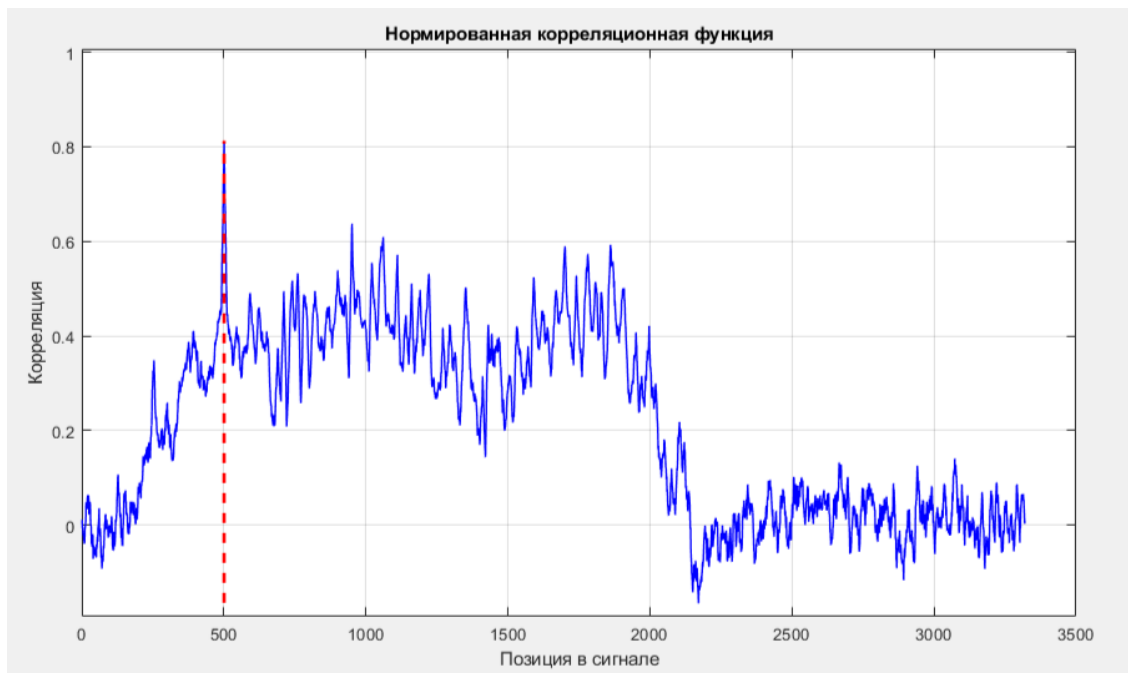


Рисунок 12 — Корреляционный пик при сигма 0,5 и N = 10

3.9 Разбор зашумленного сигнала на биты

Зная длительность в отсчетах N каждого символа разберем оставшиеся символы. Мы Берём N (5) отсчетов, усредняем их и сравниваем с пороговым значением. Пороговое значение возьмем 0.5 Добавим проверку битов на случай утери данных или если мы удалили лишний бит чтобы избежать ошибки в будущем. Итог - получили битовую последовательность до апсмплинга.

```

Длина второго массива: 1660 бит
Введите позицию для вставки (0-830): 500
σ (float): 0.3
Размер массива шума: 1660 отсчетов
корреляция=0.9290, позиция=501
Удалено 500 отсчетов
Всего битов для демодуляции: 166
Принято битов (166):
111110001101110101000010010110001010110011010010110101101110100011011
Данные+CRC: 010101100110100101101011011101000110111101110010011010010

```

Рисунок 13 — Вывод в терминал

3.10 Удаление синхропоследовательности

Удалим из полученного массива бит G-бит последовательности синхронизации.

```

1  % данные без последовательности синхронизации
2  data_crc_rx = bits_out(G+1:end);

```

3.11 Проверка CRC

Проверим корректность приема бит, для этого вычислим CRC и сравним с исходным. нам необходимо от полученной зашумленной битовой последовательности отрезать последние 7 бит и дописать вместо них нули и вычислить как в пункте 3.

```

CRC верный
Принятый CRC:
0 0 0 1 1 1 0
Вычисленный CRC:
0 0 0 1 1 1 0

```

Рисунок 14 — проверка CRC

3.12 Декодирование

Так как контрольная сумма совпала, значит полученные данные корректны и можно их преобразовывать обратно в латиницу.

```

1  % Декодирование для проверки
2  fprintf('\Проверка декодированием: "%s"\n',
    ↳ char(bin2dec(reshape(char(data_rx' + '0'), 8, []))));

```

```

Проверка декодированием: "Viktoria Pavlova"

```

Рисунок 15 — декодирование

3.13 Визуализация переданного и полученного сигнала

Визуализируем спектр передаваемого и принимаемого (зашумленного) сигналов с $N = 5$.

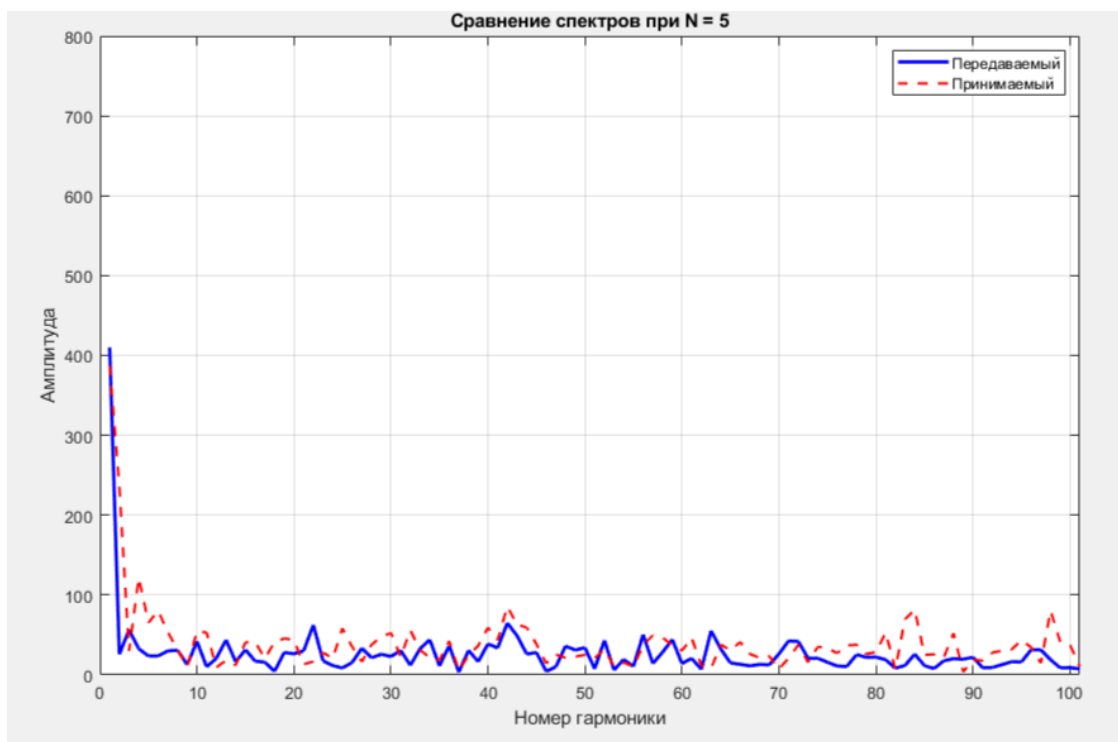


Рисунок 16 — Спектры переданного и полученного сигнала при $N = 5$

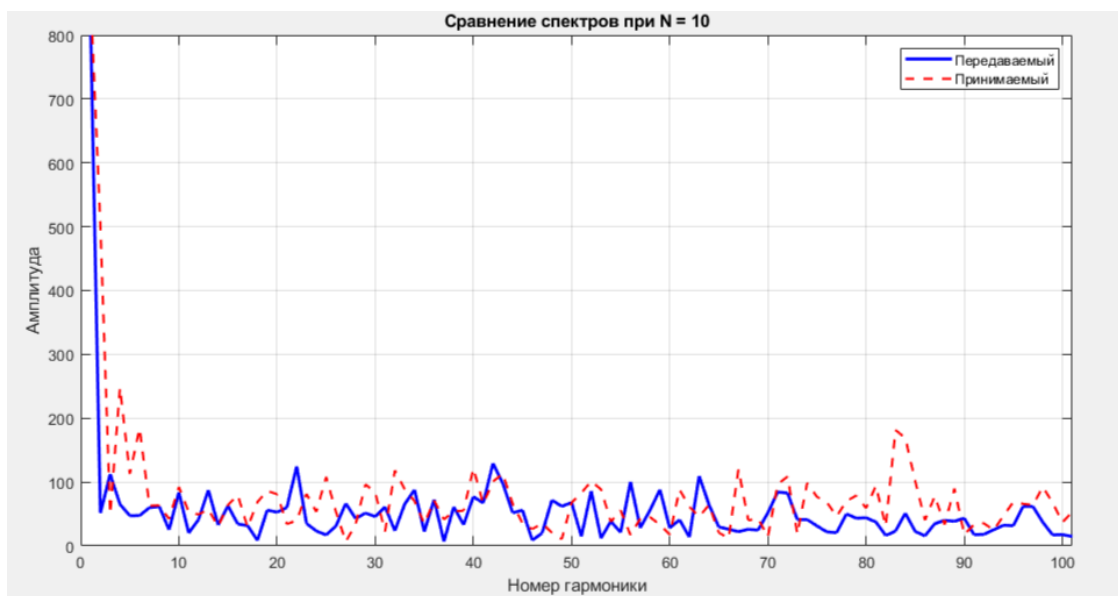


Рисунок 17 — Спектры переданного и полученного сигнала сигнала при $N = 10$

4 ВЫВОДЫ

В ходе выполнения лабораторной работы была успешно разработана и реализована система цифровой передачи данных, включающая полный цикл обработки информации от исходного текстового сообщения до его восстановления на приемной стороне. Система демонстрирует работоспособность в условиях зашумленного канала связи и обеспечивает контроль целостности передаваемых данных.

Текст успешно преобразуется в битовую последовательность с использованием ASCII. CRC-кодирование с 7-битным полиномом обеспечивает надежное обнаружение ошибок. Последовательность Голда длиной 31 бит используется для синхронизации. Реализован апсемплинг с $N=5$ отсчетов на бит.

Приемная сторона корректно выполняет синхронизацию через корреляционный анализ. Демодуляция осуществляется пороговым детектором с $P=0.5$. Система устойчива к шуму. Проверка CRC подтверждает целостность данных. При успешной передаче текст полностью восстанавливается.

Анализ спектров показывает характерные особенности сигналов: передаваемый имеет выраженную структуру, принимаемый содержит равномерный шум. Все компоненты системы работают согласованно, обеспечивая надежную передачу данных в условиях помех.

Таким образом, разработанная система наглядно демонстрирует основные принципы и методы цифровой обработки сигналов, обеспечивая надежную передачу информации по каналу с помехами. Полученные результаты подтверждают теоретические положения о возможности обнаружения и коррекции ошибок с помощью циклических избыточных кодов, а также эффективности корреляционных методов для решения задач синхронизации в системах цифровой связи.

4.1 РЕЗУЛЬТАТЫ

5 СКРИПТЫ

5.1 Скрипт RGR

```
1 name = input('Имя латиницей: ', 's');
2 surname = input('Фамилия латиницей: ', 's');
3 text = [name ' ' surname];
4 fprintf('Исходный текст: "%s"\n', text);
5
6 %% ASCIIкодер-
7 bits = dec2bin(double(text), 8) - '0';
8 res = reshape(bits, 1, []); % Преобразование в одномерный массив
9 L = length(res); % Lдлина- последовательности
10 fprintf('Длина последовательности L = %d бит\n', L);
11 fprintf('%d', res);
12
13 %% CRC
14 M = 7; % Длина CRC
15 poli = [1 1 0 1 1 1 1 0]; % Порождающий полином
16 res_copy = [res zeros(1, M)]; % Добавляем M нулей
17
18 % Проходим по битам
19 for i = 1:length(res)
20     if res_copy(i) == 1
21         % XOR с полиномом
22         for j = 0:7
23             res_copy(i+j) = xor(res_copy(i+j), poli(j+1));
24         end
25     end
26 end
27 crc = res_copy(end-M+1:end); % Берём последние M битов - это CRC
28 fprintf('\nCRCкод- (%d бит): ', M);
29 fprintf('%d', crc);
30 res_crc = [res crc]; % Добавляем CRC к данным
31
32 fprintf('\nДанные с CRC (%d бит):\n', length(res_crc));
33 fprintf('%d', res_crc);
34
35 %% последовательность Голда
36 x_reg = [0 1 1 0 0]; % x = 12
37 y_reg = [1 0 0 1 1]; % y = 19
38 m_x = generate_m_seq(x_reg, 'x');
```

```

39 m_y = generate_m_seq(y_reg, 'y');
40 gold = xor(m_x, m_y);
41 G = length(gold); % 31
42
43 fprintf('\Последовательность Голда (%d бит):\n', G);
44 fprintf('%d', gold);
45
46 tx_bits = [gold res_crc];
47 fprintf('\Полный передаваемый сигнал (%d бит):\n', length(tx_bits));
48 fprintf('%d', tx_bits);
49
50 %% апсемплинг
51 N = 10; % отсчетов на бит
52 up_signal = repelem(tx_bits, N);
53 gold_up = repelem(gold, N);
54
55 fprintf('\n\АПСЕМПЛИНГ\n', N);
56
57 fprintf('Всего отсчетов: %d\n', length(up_signal));
58 fprintf('Первые 50 отсчетов:\n');
59 for i = 1:min(50, length(up_signal))
60     fprintf('%d', up_signal(i));
61     if mod(i, 5) == 0, fprintf(' '); end
62     if mod(i, 25) == 0, fprintf('\n'); end
63 end
64
65 fprintf('\Последовательность голда после апсемплинга (%d бит):\n',
    ↪ length(gold_up));
66 fprintf('%d', gold_up);
67
68 %% создаем сторой массив и вставка
69 reserved = zeros(1, 2*N*(M+L+G));
70 fprintf('\Длина второго массива: %d бит', length(reserved));
71 position = input(sprintf('\Введите позицию для вставки (0-%d): ',
    ↪ (N*(M+L+G))));
72 start_idx = position + 1;
73
74 % Проверяем, чтобы не выйти за границы массива
75 if start_idx <= length(up_signal)
76     end_idx = min(position + length(up_signal), length(reserved));
77     reserved(start_idx:end_idx) = up_signal(1:(end_idx-start_idx+1));
78 else
79     fprintf('Ошибка: позиция выходит за границы массива\n');
80 end

```



```

81
82 %% Шум
83 sigma = input('σ (float): ');
84 noise = sigma * randn(1, length(reserved)); % Генерируем шум по нормальному
      ↳ распределению
85 fprintf('Размер массива шума: %d отсчетов\n', length(noise));
86 received_signal = reserved + noise; % Добавляем шум к сигналу
87
88 %% корреляция
89 Le = length(received_signal);
90 Lg = length(gold_up);
91 norm_corr = zeros(1, Le);
92
93 for shift = 0:Le-1
94     % Берем сегмент сигнала длиной Lg, начиная с позиции shift+1
95     start_idx_segment = shift + 1;
96     end_idx_segment = shift + Lg;
97
98     if end_idx_segment <= Le
99         % Если сегмент полностью внутри сигнала
100         segment = received_signal(start_idx_segment:end_idx_segment);
101         norm_corr(shift+1) = sum(segment .* gold_up) / sqrt(sum(segment.^2) *
      ↳ sum(gold_up.^2));
102     else
103         % Если сегмент выходит за границы - берем циклически
104         part1 = received_signal(start_idx_segment:end);
105         needed = Lg - length(part1);
106         part2 = received_signal(1:needed);
107         segment = [part1, part2];
108         norm_corr(shift+1) = sum(segment .* gold_up) / sqrt(sum(segment.^2) *
      ↳ sum(gold_up.^2));
109     end
110 end
111
112 [corr_max, sync_sample] = max(norm_corr);
113 fprintf('корреляция=%.4f, позиция=%d\n', corr_max, sync_sample);
114
115 %% Удаляем отсчеты до синхры
116 signal_clean = received_signal(sync_sample:end);
117 fprintf('Удалено %d отсчетов\n', sync_sample-1);
118
119 %% ДЕМОДУЛЯЦИЯ
120 P = 0.5;
121 total_bits = G + L + M;

```

```

122 fprintf('Всего битов для демодуляции: %d\n', total_bits);
123
124 bits_out = zeros(1, total_bits);
125
126 % Проверяем хватает ли отсчетов
127 if length(signal_clean) < total_bits * N
128     fprintf('ОШИБКА: Недостаточно отсчетов!\n');
129     fprintf('Есть: %d, нужно: %d\n', length(signal_clean), total_bits*N);
130 else
131     for i = 1:total_bits
132         start = (i-1)*10 + 1;
133         stop = start + 9;
134         five_samples = signal_clean(start:stop);
135         avg = mean(five_samples);
136         bits_out(i) = avg > P;
137     end
138 end
139 fprintf('Принято битов (%d):\n', length(bits_out));
140 fprintf('%d', bits_out);
141 fprintf('\n');
142
143 %% Разделяем на части
144 data_crc_rx = bits_out(G+1:end); % данные без последовательности
    → синхронизации
145 crc_rx = data_crc_rx(end-6:end); % CRC
146 data_rx = data_crc_rx(1:end-7); % данные без (CRC)
147 fprintf('Данные+CRC: ');
148 fprintf('%d', data_crc_rx);
149 fprintf('\n');
150
151 %% Проверка CRC
152 check_data = [data_rx zeros(1,7)];
153
154 % Вычисляем CRC для принятых данных
155 for i = 1:length(data_rx)
156     if check_data(i) == 1
157         check_data(i:i+7) = xor(check_data(i:i+7), poli);
158     end
159 end
160
161 % Получаем вычисленный CRC
162 calculated_crc = check_data(end-6:end);
163
164 % Сравниваем с принятым CRC

```

```

165 if all(calculated_crc == crc_rx)
166     fprintf('CRC верный\n');
167     fprintf('Принятый CRC: \n'); fprintf(' %d', crc_rx);
168     fprintf('\Вычисленный CRC: \n' ); fprintf(' %d', calculated_crc);
169 else
170     fprintf('Ошибка CRC\n');
171     fprintf('Принятый CRC: \n'); fprintf(' %d', crc_rx);
172     fprintf('\Вычисленный CRC: \n' ); fprintf(' %d', calculated_crc);
173 end
174
175 % Декодирование для проверки
176 fprintf('\Проверка декодированием: "%s"\n',
    ↳ char(bin2dec(reshape(char(data_rx' + '0'), 8, []))));
177
178 % Графики
179 figure();
180 stairs(res, 'LineWidth', 2);
181 title('Представление битовой последовательности');
182 xlabel('Номер бита');
183 ylabel('Значение бита');
184 ylim([-0.1 1.1]);
185 grid on;
186
187 figure();
188 stairs(tx_bits, 'b', 'LineWidth', 1.5);
189 hold on;
190 plot([31.5 31.5], [-0.1 1.1], 'r--', 'LineWidth', 2);
191 title('Сигнал: Голд + данные + CRC');
192 xlabel('Бит'); ylabel('Значение');
193 ylim([-0.1 1.1]); grid on;
194 legend('Биты', 'Граница синхр.', 'Location', 'best');
195
196 % Временной сигнал
197 figure();
198 stairs(up_signal, 'LineWidth', 1);
199 title(sprintf('Временной сигнал после апсемплинга (N=%d, всего %d отсчетов)',
    ↳ N, length(up_signal)));
200 xlabel('Номер отсчета');
201 ylabel('Амплитуда');
202 ylim([-0.1 1.1]);
203 xlim([0, (length(up_signal) + 15)]);
204 grid on;
205
206 figure();

```

```

207 stairs(reserved, 'b-', 'LineWidth', 1);
208 hold on;
209 if start_idx <= end_idx
210     plot(start_idx:end_idx, reserved(start_idx:end_idx), 'r-', 'LineWidth', 2);
211 end
212 title('Массив с вставленным сигналом');
213 xlabel('Отсчет'); ylabel('Амплитуда');
214 ylim([-0.1 1.1]);
215 xlim([0, (length(reserved) + 15)]);
216 grid on;
217
218 figure();
219 plot(received_signal, 'g-', 'LineWidth', 1);
220 hold on;
221 title('Принятый зашумленный сигнал');
222 xlabel('Номер отсчета');
223 ylabel('Амплитуда');
224 ylim([min(received_signal)-0.2, max(received_signal)+0.2]);
225 grid on;
226
227 % график корреляционной функции
228 figure();
229 plot(norm_corr, 'b-', 'LineWidth', 1);
230 hold on;
231 plot([sync_sample, sync_sample], [min(norm_corr), max(norm_corr)], 'r--',
      ↪ 'LineWidth', 2);
232 title('Нормированная корреляционная функция');
233 xlabel('Позиция в сигнале');
234 ylabel('Корреляция');
235 grid on;
236
237 %% СПЕКТРЫ
238 figure;
239
240 % Спектр передаваемого
241 subplot(2,1,1);
242 spectrum_tx = abs(fft(up_signal));
243 n_tx = 1:length(spectrum_tx)/2;
244 stem(n_tx, spectrum_tx(1:end/2));
245 title('Спектр передаваемого сигнала');
246 xlabel('Номер гармоники');
247 ylabel('Амплитуда');
248 xlim([-0.1 101]);
249 grid on;

```

```

250
251 % Спектр принимаемого
252 subplot(2,1,2);
253 spectrum_rx = abs(fft(received_signal));
254 n_rx = 1:length(spectrum_rx)/2;
255 stem(n_rx, spectrum_rx(1:end/2));
256 xlim([-0.1 101]);
257 title('Спектр принимаемого сигнала');
258 xlabel('Номер гармоники');
259 ylabel('Амплитуда');
260 grid on;
261
262 %% Генерация mпоследовательности-
263 function seq = generate_m_seq(state, type)
264     n = length(state);
265     seq = zeros(1, 2^n - 1);
266     reg = state(:)';
267
268     for i = 1:(2^n - 1)
269         seq(i) = reg(end);
270         if strcmp(type, 'x')
271             new_bit = xor(reg(3), reg(5)); % x4 XOR x5
272         else
273             new_bit = xor(reg(3), reg(5)); % y3 XOR y5
274         end
275         reg = [new_bit, reg(1:end-1)];
276     end
277 end

```

Листинг 5.1 — Код РГР на MATLAB